Custom Code POC Governance Framework

Assuring safe experimentation with custom code in the Squiz digital web content platform

Version	0.1 — Draft for director endorsement
Date	June 2026
Owner	Design engineer
Audience	Directors (assurance) and the delivery team (operations)
Purpose	To enable time-boxed, user-validated custom-code trials in web content while giving stakeholders assurance that the risk is identified, mitigated and contained.

1. On a page

This framework lets the team trial new web-design elements as custom-code **proofs of concept (POCs)** while assuring stakeholders that every trial is prioritised, user-validated, measured and time-bound to a clear end-state: **promote** it to a supported component with the DXM team, or **retire** it. Wherever a custom-code POC introduces risk, this framework sets out the mitigation already planned for it.

The lifecycle

Design	Development	Testing	Deliver	PoC (measure)	Promote ↗ or Retire ↘

Four assurances at a glance

Evidence-led	Every element is recommended by user testing and research before it is built. CSAT is then tracked at page level to confirm the change has lifted the score.
Risk-tiered	Governance scales to the material and its accessibility impact — cosmetic CSS moves fast; HTML that changes semantic structure gets a full screen-reader audit.
Safety-checked	A tiered accessibility review (screen-reader and WCAG 2.2 AA), a live register of all custom code, and a sandbox re-test before every major platform upgrade.
Time-bound	Each POC has an expiry that forces a promote-or-retire decision, so nothing temporary quietly becomes permanent.

What directors are asked to endorse

- The use of time-boxed, risk-tiered custom-code POCs as set out in this framework.
- The accessibility, register and upgrade-test gates as mandatory, not optional.
- A portfolio cap of [N] concurrent live POCs to keep the footprint contained.
- The promote-or-retire discipline, including DXM roadmap intake for promoted elements.

2. Purpose and principles

The framework balances two goals that usually pull against each other: moving quickly to test better designs, and protecting the platform from unmanaged custom code. It does this through five principles.

Risk-proportionate	Controls scale to risk, not to ceremony. A presentational tweak is not governed like a site-wide change.
Sunset by default	Custom code is provisional. The burden of proof sits on the code to earn a permanent place, not on the reviewer to justify removing it.
Evidence-led	User research recommends what we build; data confirms whether it stays.
No unmanaged tech debt	Every item is visible, owned and on a path to promotion or retirement.
Vendor boundary respected	Squiz does not support our custom code. The team owns its maintenance and any incident response for it.

3. Scope and red-lines

3.1 What counts as custom code

Category	Treatment
Custom code	Bespoke HTML and CSS — no JavaScript — built outside the supported Squiz component library to trial a new design element. In scope.
Configuration	Standard use of supported components and their settings. Out of scope.
Supported components	Out-of-the-box Squiz library components. Out of scope (this is the destination, not the trial).
JavaScript & third-party embeds	Scripts, iframes and external widgets. Out of scope — excluded by a red-line. This is what keeps the security surface minimal.

3.2 Red-lines — custom code must never

These are out of bounds regardless of tier or urgency
Include any JavaScript or depend on scripted behaviour.
Embed external third-party content (iframes, widgets or scripts).
Capture, store or transmit personal data, or handle login or session logic.
Alter global navigation, site-wide templates or master pages.
Break the screen-reader experience: remove or misuse headings and landmarks, change reading order away from the visual order, hide content from assistive technology, or drop text alternatives.
Bypass the register or the approval workflow.

4. The lifecycle and its gates

Stage	What happens	Gate / check	Owner
Design	Element prioritised from a user-tested backlog, with a documented rationale and expected benefit.	Research recommendation recorded; risk tier assigned.	UX Designer
Development	Custom component built and logged in the register with owner, tier and dependencies.	Register entry created; red-lines confirmed not breached.	Design Engineer

Stage	What happens	Gate / check	Owner
Testing	Functional and accessibility review in the sandbox environment; depth scales with tier.	Accessibility gate passed for the tier.	Design engineer + reviewers (tbd)
Deliver	Deployed to production with a rollback plan and baseline metrics captured.	Go-live approval per tier; baseline recorded.	Developer / PM
PoC (measure)	Live for the time-box. CSAT uplift and behavioural metrics tracked against baseline and pre-agreed exit criteria.	Time-box expiry forces a decision.	UX Design / Student Voice/ Research teams in ESG
Decision	Promote / Scale with DXM, or Retire & change page.	DXM roadmap entry, or migration to a supported component / page retirement.	Director + DXM + Design Engineer

The two outcomes

Promote / Scale with DXM	Retire & change page
CSAT uplift meets the pre-agreed bar. DXM converts the element to a supported component, or adds it to the supported inventory and the regression test set.	No significant uplift. The page is migrated to a supported component or retired. A successful experiment that avoided permanent debt.

5. Risk tiering

Tiers are set by **material and accessibility impact**: CSS versus HTML, whether semantic structure or reading order changes, and the screen-reader impact. Reuse across pages does not raise the tier — it widens the representative-page testing at upgrades (section 8).

Tier	Triggers	Governance required	Approver
Tier 1 — Cosmetic CSS	CSS only; cosmetic styling — colour, typography, spacing, decoration. No change to layout order, visibility or contrast below AA.	Accessibility self-check (contrast to AA; nothing hidden from assistive tech; visual order matches DOM order); peer review; register entry.	Manager, Student Comms
Tier 2 — CSS layout / additive HTML	CSS that changes layout, reading order or visibility; or HTML that adds or restyles content without altering semantic structure.	Accessibility review — screen-reader pass on reading order, hidden content and text alternatives; named owner; rollback plan.	Manager, Student Comms DXM Lead
Tier 3 — Structural HTML	HTML that changes semantic structure — headings, landmarks, lists, tables or figures that screen readers announce.	Full accessibility audit (NVDA / VoiceOver; headings & landmarks; reading order; WCAG 2.2 AA); accessibility sign-off; rollback plan; shortest time-box.	Accessibility lead + DXM Lead, Manager, Student Comms

6. Risk and mitigation

Wherever a custom-code POC introduces risk, there is a mitigation already planned. This is the reassurance the framework provides.Before we begin, below is a draft definition of a custom block:

Custom code: a raw html code that can be added for Matrix CMS to produce custom UX elements on any given page by embedding into either a 'raw code' component or some other fields, such as tags.

Risk	Mitigation already planned	When it applies
Custom HTML changes semantic structure and breaks the screen-reader experience.	Tiered accessibility review; screen-reader pass on headings, landmarks, reading order and text alternatives; a11y error count tracked as a guardrail metric.	Tier 2–3, before go-live
CSS hides content from assistive tech, drops contrast, or reorders reading order.	Tier 1 accessibility self-check: contrast to WCAG 2.2 AA, nothing hidden from assistive tech, and visual order matches DOM order.	Every POC
Security or data exposure.	Handled by design, not by review: JavaScript, external embeds and data capture are red-lines, so the scripting and injection surface is removed.	By scope (all POCs)
A future platform / library upgrade silently breaks a custom page.	Live register of all custom-code pages; mandatory sandbox re-test before every major upgrade — three representative pages per shared component is sufficient, not every page.	Every major Squiz upgrade
"Temporary" custom code becomes permanent tech debt.	Time-box with a forced promote-or-retire decision; pre-committed exit criteria; portfolio cap on concurrent POCs. A custom code block can be written in a 'raw code' component that will only be used as a component on the page, or in certain fields such as tags, link type components for adding URLs. A custom code block will not be used outside of a block within a page	Every POC
A live POC fails or degrades the page.	Rollback plan captured at deploy; documented migration path to a supported component.	Tier 2–3
Ownership is lost ("orphaned" code).	Named accountable owner per register entry; RACI; ownership re-confirmed at each upgrade check.	Every POC

7. Measurement

User testing and research is what **recommends** an element for trial — that is the decision to build. CSAT is then measured **at page level** to see whether the changed elements have lifted the score. We pair it with behavioural and guardrail metrics so the case never rests on a single soft signal.

Metric type	Examples	Purpose
Research basis	Usability testing and user-research findings.	Justifies building the element — the "why".
Primary signal	Page-level CSAT uplift versus baseline.	Confirms the element improved satisfaction.
Behavioural	Task completion, time-on-task, support-ticket / contact volume, drop-off / bounce.	Corroborates CSAT with observed behaviour.

Metric type	Examples	Purpose
Guardrail	Page load / performance, accessibility error count, error rates.	Ensures the POC does not make the page worse.

Baseline is captured before deploy. Exit criteria — the CSAT bar and the minimum sample or run-time — are agreed **before** the test starts, not after the data lands, and are tied to the time-box.

8. The live register

A single live register in Confluence is the backbone of the framework. Every custom-code element is recorded before go-live and maintained for its life.

Field	Purpose
ID & element	Unique reference and the page(s) it appears on.
Purpose & research link	Why it exists and the testing that recommended it.
Owner	The named, accountable person — no orphans.
Risk tier	Tier 1 / 2 / 3 and the trigger that set it.
Dependencies	Libraries, scripts or components it relies on.
Deploy date & time-box expiry	When it went live and when the decision is forced.
Support status	Custom · promoted · retired.
Rollback / migration plan	How to revert or move to a supported component.
Representative test pages	For a shared component, the (up to three) pages re-tested at each major upgrade.
Last upgrade-check date	When it was last re-tested against a platform upgrade.

Upgrade check. Before every major Squiz upgrade, a nominated owner re-tests the custom-code components in the sandbox environment and confirms nothing is broken before the upgrade is promoted to production. Where the same custom code appears on multiple pages, testing three representative pages per component is sufficient — full page-by-page testing is not required.

9. RACI

R Responsible · **A** Accountable · **C** Consulted · **I** Informed · **–** not involved

Stage	Director	PM	Designer/Dev	DXM	Access.	SME
Design	I	A	R	C	C	C
Development	I	A	R	C	C	–
Testing	I	A	R	C	R	C
Deliver	I	A	R	C	C	I
PoC (measure)	I	A/R	R	I	–	I
Promote / Retire	A	R	C	R	I	I

10. Approval workflow and tooling

The approval trail is recorded, not informal. The recommended split across the Atlassian stack:

Tool	Role
Jira	A “Custom Code POC” issue type with statuses mirroring the lifecycle (Design → … → Promote / Retire) and transition approvals. Gives an auditable, reportable sign-off trail.
Confluence	Home of this framework, the live register and the RACI. Register rows link to their Jira issue.
Dashboard	A Jira dashboard or a Confluence chart macro for tech-debt visibility (section 11).

Lighter alternative. A Confluence-only option (page templates with sign-off columns and @mentions, or a Comala-style page workflow) is viable but less auditable than a Jira workflow.

11. Tech-debt dashboard

Debt you can see is debt you can manage. A single dashboard makes the custom-code portfolio visible at a glance and shows:

- Count of live custom-code POCs against the agreed portfolio cap.
- Each item's tier, owner and age versus its time-box (RAG status).
- Promote / retire / overdue-for-decision status.
- Items pending an upgrade re-test.
- Cumulative custom-code (debt) trend over time.

12. Decision sought

Directors are asked to endorse:

1. the use of time-boxed, risk-tiered custom-code POCs as set out here;
2. the accessibility, register and upgrade-test gates as mandatory;
3. a portfolio cap of [N] concurrent live POCs; and
4. the promote-or-retire discipline, including DXM roadmap intake for promoted elements.
5. conduct a ne as a one-off exercise to know exactly how many pages have custom code blocks
6. regularly audit (room for automation) currently used code-blocks within a 3 month cycle and cross-reference with the live register for any discrepancy

Appendix A — POC intake template

Completed at the Design stage; becomes the register entry on approval.

- Element name and page(s).
- Research / testing reference and rationale.
- Expected benefit.
- Proposed risk tier and the trigger that sets it.
- Material: cosmetic CSS, CSS layout / additive HTML, or structural HTML (this sets the tier).
- Dependencies.
- Proposed time-box.
- Baseline metrics.
- Exit criteria (CSAT bar, minimum sample / run-time).
- Rollback / migration plan.
- Owner.

Appendix B — Glossary

POC	Proof of concept — a time-boxed trial of a new design element.
DXM	The team that owns the supported component library and product roadmap.
CSAT	Customer satisfaction score, measured at page level here.
WCAG 2.2 AA	Web accessibility standard; the compliance target under the DDA.
AT	Assistive technology — screen readers (e.g. NVDA, VoiceOver) and similar tools.
RAG	Red / Amber / Green status used on the dashboard.
Time-box	The fixed period after which a promote-or-retire decision is forced.